# OutSentimented BERT Based Financial News Sentiment Intelligence

Muhammad Haris, FA23-BBD-094 Sobia Rauf, FA23-BBD-133

*BS Business Data Analytics, Department of Management Sciences*

*MGT488 Advanced Deep Learning*

***Abstract*—Financial news arrives faster than analysts can consistently classify it. OutSentimented proposes one decision-support module that receives an English financial sentence and returns a positive, neutral, or negative label with a confidence score. The Week 8 work verifies Financial PhraseBank, removes five exact duplicate sentences, and produces leakage-free 70/15/15 partitions from 3,448 unique records. The later baseline will use TF-IDF with multinomial logistic regression. The advanced model will fine-tune the KerasHub bert_base_en_uncased preset with a 128-token input. Evaluation will emphasize macro F1, per-class performance, and error analysis because the negative class is small but operationally important. This proposal defines the business value, reproducible data pipeline, architecture, evaluation plan, limitations, ethics, and semester schedule. It does not report trained-model results.**

***Index Terms*—BERT, financial news, sentiment classification, Financial PhraseBank, decision support**

## I INTRODUCTION

Financial analysts monitor earnings releases, profit warnings, merger announcements, regulatory actions, and market updates across many sources. Manual review is slow and individual judgments may vary. A first-pass classifier can help an analyst prioritize adverse developments and organize a news queue, while leaving interpretation and investment decisions to a human. The proposed system addresses Track C of the course guidelines by adapting a pre-trained transformer to a business sentiment task [1], [7].

OutSentimented has one functional scope. It accepts one English financial-news sentence and returns one label from positive, neutral, or negative together with a confidence score between zero and one. The module will not recommend trades, predict prices, summarize articles, or ingest live feeds during this semester. This boundary keeps the project testable and aligns the model output with the labels in Financial PhraseBank.

The objectives are to build a reproducible text pipeline, compare a traditional sparse-text baseline with a fine-tuned BERT classifier, measure performance on a held-out test set, and identify class-specific failure patterns. Week 8 establishes the proposal and data evidence. Model fitting remains future work, so this document reports no baseline accuracy, BERT score, or trained weights.

## II BUSINESS VALUE AND MODULE SCOPE

The intended user is a financial analyst or monitoring team. Negative items can be surfaced for earlier review, neutral announcements can be separated from events with an apparent financial direction, and consistent first-pass categorization can reduce repetitive screening. Confidence supports triage: a low-confidence item can be routed for immediate manual review rather than treated as a reliable automated conclusion.

The module contract is classifier(sentence) → {label, confidence}. Input validation will reject empty text and document the English-only constraint. The output label will match the training taxonomy, and confidence will be the selected class probability after any calibration decision. Predictions are analytical aids, not investment advice. Production integration, streaming, user authentication, and portfolio actions are outside the project scope.

Three research questions guide later experiments. First, how much macro F1 does BERT add over the TF-IDF baseline under one fixed split? Second, which classes gain or lose recall, especially the minority negative class? Third, which linguistic patterns remain difficult after fine-tuning? The project will answer these questions with held-out metrics and reviewed errors rather than qualitative demonstrations alone.

## III DATASET VERIFICATION AND EDA

### A Source and license

Financial PhraseBank contains English sentences from financial news annotated from an investor perspective [1]. The project uses the sentences_75agree configuration, which retains items with at least 75 percent annotator agreement. The Hugging Face dataset card documents 3,453 records for this configuration and no predefined train, validation, or test split [2]. The downloaded archive has SHA-256 digest 0e1a06c4900fdae46091d031068601e3773ba067c7cecb5b0da1dcba5ce989a6. The data is licensed under CC BY-NC-SA 3.0; source text is downloaded during notebook execution and is not committed to the MIT-licensed code repository.

### B Quality checks and class balance

The executed notebook finds no missing or blank sentences and no missing labels. Five sentences occur twice, always with neutral labels. Removing one copy from each pair leaves 3,448 unique sentences: 2,141 neutral, 887 positive, and 420 negative. The negative class is 12.18 percent of the deduplicated set, so accuracy alone could hide weak performance on the most important triage category.

| Split | Negative | Neutral | Positive | Total |
|---|---|---|---|---|
| Train | 294 | 1,498 | 621 | 2,413 |
| Validation | 63 | 321 | 133 | 517 |
| Test | 63 | 322 | 133 | 518 |

**TABLE I DETERMINISTIC STRATIFIED PARTITIONS**

### C Partitioning and exploratory findings

After deduplication, the notebook creates stratified partitions with seed 42. It assigns 2,413 records to training, 517 to validation, and 518 to testing. Exact sentence sets have zero overlap. The split files are generated locally and ignored by Git. Class charts, sentence-length statistics, representative samples, and class-level bigrams are retained as executed notebook outputs. Whitespace length is descriptive; the Week 12 pipeline will also measure WordPiece length before confirming truncation behavior.

Leakage controls extend beyond the initial split. TF-IDF vocabulary fitting, scaling decisions, class-weight selection, BERT checkpoint choice, and any probability calibration will use only training and validation data. The test set will not be inspected for model selection. Derived files will retain a split name but not become a separately published dataset because the source license governs their use.

## IV SUCCESS CRITERIA

Milestone success has separate reproducibility and modeling criteria. Reproducibility requires a fresh Python 3.10 environment to install pinned dependencies, download the verified archive, execute every notebook cell, and regenerate the same row counts and split sizes. Modeling success requires both planned classifiers to run on the same partitions and produce complete test metrics. The advanced model will be preferred only if its macro F1 and per-class results justify its added cost.

No single target score is promised before experiments. A useful result must instead establish a transparent comparison, preserve minority-class visibility, and state uncertainty. If BERT does not exceed the baseline, the project will report that outcome and analyze whether data size, tuning limits, or domain ambiguity explain it. A reproducible negative result is more valuable than an unsupported performance claim.

## V PLANNED METHODOLOGY

### A Comparison baseline

The baseline will combine TF-IDF features with multinomial logistic regression. Candidate settings will include unigram and unigram-plus-bigram vocabularies, minimum document frequency, maximum feature count, regularization strength, and class weighting. Only training data will fit the vocabulary and estimator. Validation macro F1 will guide selection. The untouched test split will be evaluated once after the baseline and BERT decisions are fixed.

### B Advanced model

The advanced classifier will use TensorFlow, Keras, and KerasHub. BertTextClassifier.from_preset will load bert_base_en_uncased with three output classes [4]. The BERT base architecture has about 109 million parameters [3]. The preprocessor will tokenize raw strings, add special tokens, create segment and padding tensors, and pack inputs to a maximum length of 128 [4].

Fine-tuning will start with a small batch size appropriate for a 16 GB M1 MacBook Pro. Planned trials will vary learning rate, epoch limit, dropout if needed, and whether the backbone is initially frozen. Early stopping will monitor validation macro F1 or validation loss and restore the best checkpoint. The search will remain modest and documented so comparisons are reproducible rather than selected from unreported trials.

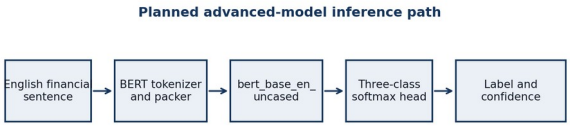



Fig. 1. Planned advanced-model inference path.

## VI EVALUATION PLAN

The final comparison will report accuracy, macro precision, macro recall, macro F1, per-class precision, recall, and F1, plus a confusion matrix. Macro F1 is the primary selection metric because it gives equal weight to negative, neutral, and positive performance. Accuracy remains useful for overall correctness but will not determine the preferred model by itself.

Error analysis will sample false negatives, false positives, and neutral confusions. Review categories will include negation, mixed events, comparative language, numeric changes, long sentences, ambiguous financial impact, entity-specific phrases, and annotation uncertainty. Confidence distributions will be inspected for overconfident errors. If calibration is poor, temperature scaling or a documented abstention threshold may be evaluated on validation data without changing the held-out test labels.

The comparison will use identical deduplicated partitions for both models. Training-time observations, inference latency on local hardware, and model size will be recorded as secondary operational measures. Statistical claims will remain limited by the single dataset and test split; no result will be generalized to live markets without additional external validation.

Every reported metric will identify its split, model version, preprocessing settings, and decision threshold. The final notebook will save predictions for error review without adding source sentences to a public release. Confusion matrices will show raw counts as well as normalized rates so readers can distinguish class size from error proportion.

## VII IMPLEMENTATION AND COMPUTING PLAN

Repository setup targets Python 3.10 with pinned EDA dependencies. The Week 8 notebook downloads and verifies the source archive, generates ignored local partitions, and stores executed outputs in version control. The classifier interface is represented in the package as typed input-output contracts. Later modules will separate preprocessing, training, evaluation, and inference so the notebook demonstrates use rather than holding the only implementation.

BERT training is reserved for the local M1 MacBook Pro with 16 GB memory. Before Week 12, a separate compatible environment will validate the current TensorFlow and Metal plug-in combination on a small tensor and one short training step [5]. Small batches, dataset prefetching, checkpointing,

and early stopping will control memory and runtime. CPU execution remains a slower fallback if Metal compatibility is unstable. Environment versions and hardware observations will be recorded with each experiment.

## VIII RISK AND REPRODUCIBILITY CONTROLS

### *A Modeling risks*

The largest modeling risk is learning a strong neutral default while missing negative events. Stratification prevents split-level class collapse but does not solve the imbalance itself. Baseline trials will compare unweighted and balanced logistic regression. BERT trials may use class weights only when validation macro F1 and negative recall improve without an unacceptable rise in false alarms. Oversampling is not the default because repeated minority examples could increase memorization in this small dataset.

Overfitting is another risk. The advanced model has far more parameters than training examples, so a high training score would not establish useful generalization. Early stopping, short runs, limited tuning, and fixed validation data will constrain the search. The team will record every completed configuration considered in the comparison, including unsuccessful runs, rather than presenting only the best checkpoint without context.

### *B Execution risks*

Local training may be limited by unified memory, TensorFlow and Metal version compatibility, or long CPU fallback times. A preflight test will confirm package imports, device visibility, one forward pass, and one small optimization step before full training. Out-of-memory failures will be addressed by reducing batch size and using shorter test runs before changing the model. If the Metal path produces unstable behavior, the same configuration will run on CPU and the added time will be documented.

The EDA pipeline also depends on a remote dataset archive. Reproducibility is protected by a stable source URL, the expected SHA-256 digest, an explicit archive member name, and assertions for row count and label values. A changed or unavailable archive will fail visibly rather than silently produce different data. The ignored local copy can support reruns on the same machine without turning the Git repository into a dataset mirror.

### *C Experiment records*

Each modeling run will record the Git commit, Python and library versions, random seed, split files, feature or tokenizer settings, training duration, best epoch, validation metrics, and checkpoint path. Results tables will be generated from saved machine-readable records when implementation begins. This prevents manual transcription from becoming the only evidence for a reported score and makes later changes traceable.

Final test evaluation is a separate decision gate. Model and threshold choices will be frozen from training and validation evidence before test predictions are generated. If a defect is later found in data handling, the correction and reason for rerunning the test will be documented. The repository history will retain the earlier state rather than overwriting the sequence of decisions.

## IX LIMITATIONS AND ETHICS

Financial PhraseBank is relatively small, drawn from a specific news domain, and imbalanced toward neutral language. Its investor-perspective labels compress context into one sentence and three categories. A sentence may contain multiple events, depend on prior text, or change meaning across firms and market conditions. Annotation agreement reduces ambiguity but does not eliminate it. BERT may also learn publication patterns, company names, or frequent phrases instead of general financial meaning.

The dataset license limits use to noncommercial conditions and requires attribution and share-alike treatment for adapted data. Repository code can use MIT terms, but those terms do not replace the dataset license. The project will not expose source rows as a repackaged dataset. Reports will cite the original authors and hosting record. Predictions will be presented as screening support, never personalized advice or evidence of future returns.

Bias checks will compare per-class behavior and inspect whether errors concentrate around sectors, locations, currencies, or named entities when sample size permits. The team will document uncertainty, preserve the test set, and avoid tuning against final results. A user-facing implementation should provide confidence, model version, and a route for human review rather than silently acting on a label.

## X TIMELINE AND DELIVERABLES

| Stage | Planned work | Evidence |
|---|---|---|
| Week 8 | Proposal, source verification, EDA, split audit | DOCX, PDF, executed notebook |
| Week 12 | TF-IDF baseline, preprocessing, initial BERT run | Curves, metrics, checkpoints |
| Week 15 | Tuning, final test, error analysis | Code, model, benchmark tables |
| Exam week | IEEE report and live demonstration | Final report and presentation |

Week 8 is complete when the proposal renders correctly, the notebook reruns without failed cells, dataset checks and split assertions pass, the README reproduces setup, and repository metadata is correctly attributed. Week 12 begins only after TensorFlow Metal validation and does not alter the held-out test data. Week 15 will add trained artifacts and measured results through separate commits.

## XI WEEK 8 CONCLUSION

OutSentimented defines a narrow, testable sentiment-classification module with a clear analyst-support use case. The Week 8 package verifies the data source, documents license constraints, removes exact duplicates before partitioning, and produces reproducible stratified splits with no sentence overlap. It also specifies a credible comparison between TF-IDF logistic regression and a BERT classifier without claiming results that have not been measured. The next milestone is baseline implementation and validation of the local BERT training environment.

## REFERENCES


[1] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," Journal of the Association for Information Science and Technology, vol. 65, no. 4, pp. 782–796, 2014.

[2] P. Takala, "Financial PhraseBank dataset card," Hugging Face, 2026. [Online]. Available: https://huggingface.co/datasets/takala/financial_phrasebank. Accessed: Oct. 28, 2026.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[4] Keras Team, "BertTextClassifier," KerasHub API documentation, 2026. [Online]. Available: https://keras.io/keras_hub/api/models/bert/bert_text_classifier/. Accessed: Oct. 28, 2026.

[5] Apple Inc., "Accelerated TensorFlow on Mac," Apple Developer, 2026. [Online]. Available: https://developer.apple.com/metal/tensorflow-plugin/. Accessed: Oct. 28, 2026.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] Department of Management Sciences, "MGT488 Advanced Deep Learning semester project guidelines, milestones, and evaluation rubrics," course handout, 2026.